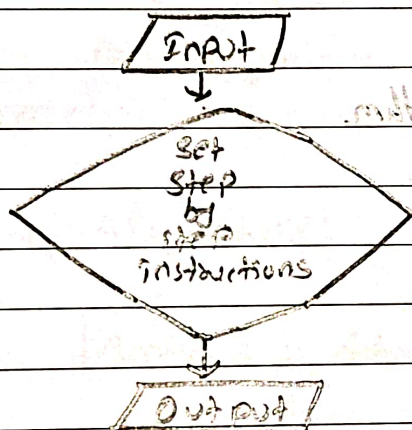


Basic concepts of Algorithms:-

Algorithm

- An algorithm is a set of step by step instructions for solving a problem or completing a task. It tells us exactly what to do and how to get the final result.

Computer uses Algorithms to help them make decisions, process data or perform action automatically.



Characteristics of an algorithm:

- 1) Clear and unambiguous
- 2) Input
- 3) Output
- 4) Infinite
- 5) Effectiveness
- 6) Feasibility
- 7) Language interpretation.

Types of Algorithms:

- i) Brute force Algorithm
- ii) Greedy Algorithm
- iii) Divide and Conquer Algorithm
- iv) Dynamic programming Algorithm
- v) Back tracking Algorithm
- vi) Recursive Algorithm
- vii) Searching Algorithm
- viii) Sorting Algorithm
- ix) Hashing Algorithm
- x) Machine Language Algorithm.

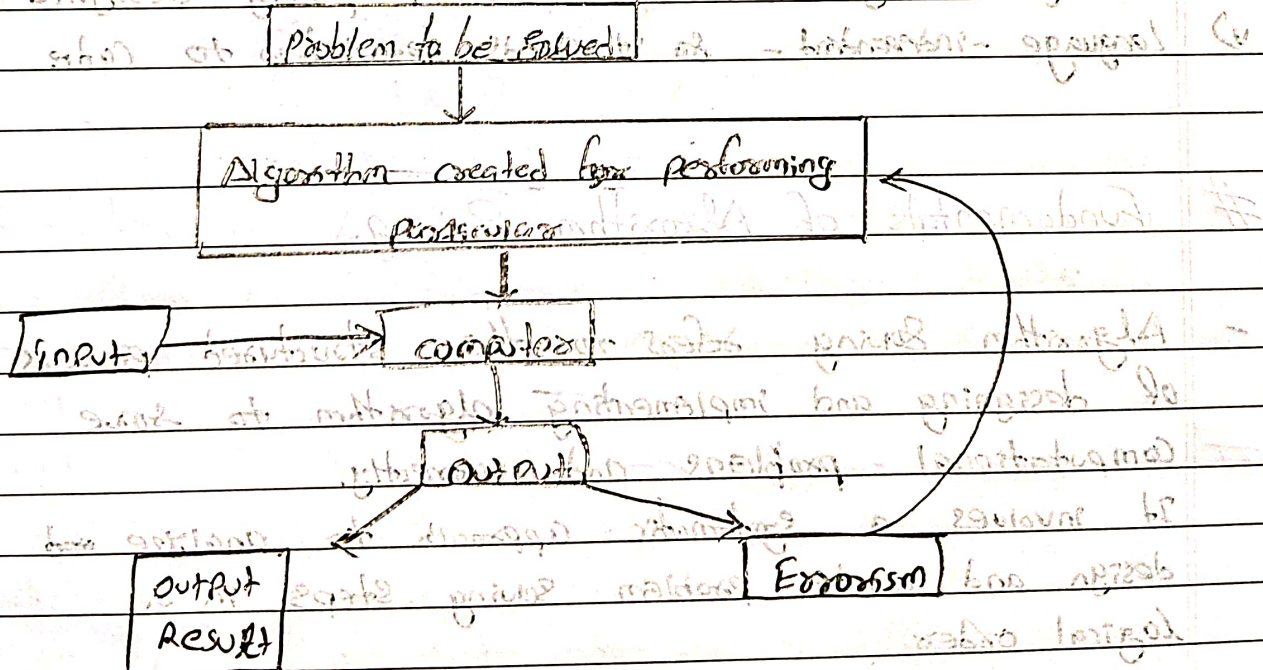
Basic e.g (a+b): Algorithm

- Step 1: Start
- Step 2: Create a variable like a and b.
- Step 3: input a first number.
- Step 4: input b second number.
- Step 5: Add a + b
- Step 6: Print a + b
- Step 7: Stop

Write an algorithm to find the greatest number among three.

- Step 1: Start
- Step 2: Input a first number
- Step 3: Input b second number
- Step 4: Input c third number
- Step 5: Check if condition $a > b$ and $a > c$ is satisfied or not
- Print a greater
- Step 6: else if $b > a$ and $b > c$
- Print b greater
- Step 7: else print c greater
- Step 8: Stop

Notation of Algorithms:



method, all parameters

function & input & output

choice and operators

break the problem into sub-problem

also have alternative

H.W.
80/07/2025

Advantages of Algorithms

- 1) Clarity - Steps are clear and well-defined
- 2) Reusability - Can be used in multiple problems
- 3) Efficiency - Helps optimize time and space
- 4) Debug-friendly - Easier to find and fix issues
- 5) Foundation for coding - Blueprint before implementation

Disadvantages of Algorithms

- 1) Time-consuming to write - Especially for complex problems
- 2) Not always practical - May not cover all real-world cases
- 3) Needs good logic skills - Not beginner-friendly sometimes
- 4) Can get messy - Hard to follow if poorly designed
- 5) Language-independent - So it needs translation to code

Fundamentals of Algorithm Solving:

- Algorithm solving refers to the structured process of designing and implementing algorithm to solve computational problems and correctly.

It involves a systematic approach to analyze and design and write problem solving steps in a logical order

- 1) Understanding the problem
- 2) Identify the input & output
- 3) Choose and appropriate strategy
- 4) Break the problem into sub problem
- 5) Design the algorithm pseudo code, flow-chart.

vi) Analyze the algorithm.

vii) Implement the algorithm.

viii) Test and debug.

Ans

What could be the time complexity of for loop and do-while loop.

→ The time complexity does not depend on type of loop. It is about how many times the loop runs based on the input size (like n).

~~For~~ ex: (do-while)

ex: for

```
int i = 0;
do {
    // code
    i++;
} while(i < n);
```

```
for (i = 0; i < n; i++)
{
    // code
}
```

Here these will run ' n ' times, assuming n is the upper bound and each iteration takes constant time.

Hence Time complexity:

- if loop runs n times $\rightarrow O(n)$
- if loop is a nested loop $\rightarrow O(n^2)$ or worse
- if loop have the input everytime $\rightarrow O(\log n)$

Important problem type:

→ The most important problem type are

- i) Sorting
- ii) Searching
- iii) String processing
- iv) Graph problem
- v) Combinatory problem
- vi) Geometrical problem
- vii) Numerical problem

Fundamental of Algorithm Framework:

The efficiency of an algorithm can be in term of time and space. The algorithm efficiency can be analyze by the following way:-

- i) Analyzing framework
- ii) Asymptotic Notation and its Properties
- iii) Mathematical Analysis for recursive algorithm
- iv) Mathematical Analysis for non recursive algorithm.

Analyzing Framework:-

→ There are two kinds of efficiency to analyze the efficiency of any algorithm they are:-

- i) Time efficiency
- ii) Space efficiency

Time efficiency

- It indicates how fast the algorithm runs

Space efficiency

- Indicate how much extra memory it use.

The algorithm analyze the framework consists of the following

- i) measuring the input size
- ii) unit ~~for~~ for measuring run time
- iii) order of growth
- iv) worst case, Base case, Average case efficiency
- v) Recapitulation of the analyzing framework.

Some important complexities:-

n	$\sqrt{n}$	$\log n$	n	$n \log n$	n^2	n^3	2^n	$n!$
1	1	0	1	0	1	1	2	1
2	1.4	1	2	2	4	4	4	2
4	2	2	4	8	16	64	16	24
8	2.8	3	8	$2.4 \cdot 10^1$	64	$5.1 \cdot 10^2$	$2.6 \cdot 10^3$	$4.0 \cdot 10^4$
10	3.2	3.3	10	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
16	4	4	16	$6.4 \cdot 10^1$	$2.6 \cdot 10^2$	$4.1 \cdot 10^3$	$6.5 \cdot 10^3$	$2.1 \cdot 10^{13}$
10^2	10	6.6	10^2	$6.6 \cdot 10^4$	10^4	10^6	10^6	$1.9 \cdot 10^{30}$
10^3	31	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	10^2	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}	very big	
10^5	$3.2 \cdot 10^2$	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}	computation	
10^6	10^3	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

1) Measuring the input size

→ An algorithm efficient is defined as a function of same parameter 'n' indicates the algorithm input size. In most selections such as parameter is quite strongly forwarded.

Example

- it will be the size of the list for problem of sorting, searching

Loop example

```
for (i=0; i<10; i++)
```

{

ideal case formula for measuring input size = $b = (\log_2 n) + 1$

Unit of measuring run time

Some standard unit of time measuring such as second or millisecond as 800 on.

Example: may get the target to the area covered may not

It can be used measuring the run time of a program often implement the algorithm.

{ Drawbacks }

- Depends on the speed of particular computer
- Depend on the quality of a program implemented the algorithm
- The compiler used in generating the machine code.
- The drawback of clocking actual run-time of program.

{ +, -, *, % }

III) Order of growth:

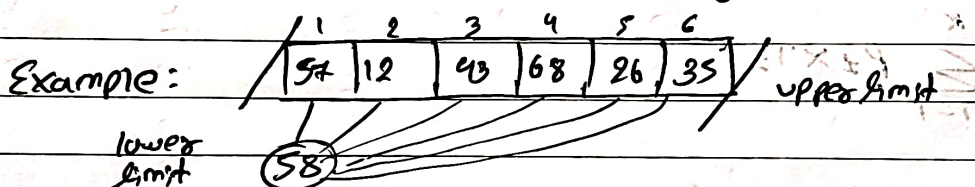
Refer to previous page table:-

- A difference in running time on small input is ~~known~~ not what really distinguish efficiency of algorithm or efficient one

Example: voting system (less data = ~~less~~ or may or may not actual value)

IV) Worst case

- In the worst case we analysis and calculate the "upper bound" on running time of an algorithm. we must know the case that cause maximum number of operation to be execute. we also called it Big - O notation.

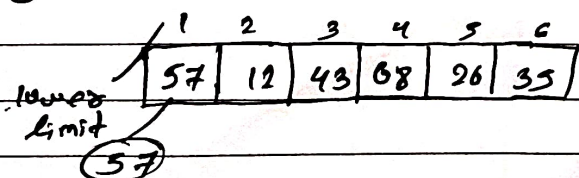


- It takes maximum time

Best case:-

- in the best case analysis we calculate lower bound on running time of an algorithm. we must know the case that cause minimum number of operation to be execute. we also called it Big - Ω (Omega).

- This algorithm take minimum time



Average case Analysis?

→ In average case analysis, we take all possibility inputs and calculate computing time for all of the input. Sum all the calculated value and divide the sum by total number of input. we must know (prediction) distribution of case. It is denoted by P_i that is (H)

$$\text{Average} = \frac{n_1 + n_2 + n_3 + \dots + n_{\infty}}{n_{\infty}}$$

$$A(n) = \sum_{i=1}^k P_i \times T_i$$

$$= P_1 \times T_1 + P_2 \times T_2 + \dots$$

$$= \sum_{i=1}^k P_i \times T_i$$

1	2	3	4	5	6	7	8	9	10
28	28	28	28	28	28	28	28	28	28

time

it takes maximum time

Recursive algorithm:

Recursive is a technique used in computer science to solve big problem by breaking them into small, similar problem. The process in which a function call itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

A recursive algorithm is an algorithm that use recursion to solve a problem.

Types of Recursion:

- i) Direct recursion
- ii) In-direct recursion
- iii) Head recursion
- iv) Tail recursion

Application of recursion

- i) Tree and Graph traversal
- ii) Dynamic Programming
- iii) Divide and Conquer
- iv) Backtracking
- v) Combinatory